# Deeparture V2 - Six-Phase Development Roadmap

**Current developer roadmap**

**Guiding principle:** Design the extension point now; implement the feature in the correct phase; add future integrations as adapters rather than rebuilding the core.

**Current roadmap for developer reference**

This document defines the six-phase plan for Deeparture V2. The intention is to preserve a clear sequence of work, avoid rebuilding completed foundations, and keep future integrations possible without repeatedly changing the core architecture.

**Guiding principle:** Design the extension point now; implement the feature in the correct phase; add future integrations as adapters rather than rebuilding the core.

## Phase 1 - Foundation & Staging

### Objective

Establish the standalone V2 technical foundation and staging environment without disrupting the legacy Deeparture system.

### Core scope

- Standalone V2 codebase and repository.
- Backend/API and frontend foundations.
- MongoDB Atlas staging database.
- Render staging deployment.
- Authentication and basic role separation.
- Organisation / operator ownership model.
- Liveaboard and resort data foundations.
- Itinerary, departure, pricing and availability foundations.
- Automatic public pages from database records.
- Basic search and filtering.
- Operator claim foundations.
- Enquiry and email foundations.
- Seed/demo data and verification tools.

### Status

Phase 1 is substantially complete and is the baseline for all later work.

### Important rule

Phase 1 code should be extended, not replaced. The exact branch and commit deployed to Render should always be recorded.

## Phase 2 - Admin V2 & Catalogue Automation

Phase 2 is divided into **Phase 2A** and **Phase 2B** to keep implementation manageable.

### Phase 2A - Core Admin V2

### Objective

Give Deeparture a practical internal administration system so catalogue records, organisations, listings, imports, availability and enquiries can be managed without depending on operators to enter everything themselves.

### Core scope

- Dedicated Admin V2 interface.
- Admin dashboard.

- Liveaboard catalogue management.
- Resort catalogue management.
- Create / edit / preview / publish / unpublish / restrict / archive.
- Draft and approval states.
- Operator organisation management.
- Unclaimed / invited / claimed organisation states.
- Operator claim invitation workflow.
- Facilities, dietary, diving, accessibility and other structured attributes.
- Structured destinations and ports.
- Responsive desktop, tablet and mobile admin screens.
- Existing public pages continue to be generated automatically from database records.

## Import architecture

Phase 2A uses a normalized import pipeline:

```
source adapter -> mapping/normalization -> Deeparture normalized schema -> validation -> matching
-> preview -> confirmation -> catalogue service -> database
```

Supported initially:

- XLSX.
- CSV.

Future sources must plug into the same pipeline rather than writing directly to MongoDB.

## Availability architecture

Phase 2A establishes a source-independent **Normalized Availability Service**.

Availability may initially come from:

- Admin/manual entry.
- CSV/XLSX import.

Future sources may include:

- Operator Portal.
- Supplier API.
- Webhooks.
- JSON/XML feeds.
- Google Sheets.
- SFTP.
- External booking-system connectors.
- Website ingestion.

Public/search layers should consume resolved availability without needing to know which source supplied it.

## Enquiry workflow

Preserve the Admin-first workflow:

```
customer enquiry -> customer confirmation -> Admin review -> Admin approval -> operator
notification -> operator response -> Admin confirmation
```

Do not send new customer enquiries directly to an operator before Admin review.

## Implementation approach

Phase 2A is built on the existing Phase 1 code. Working Phase 1 foundations such as pricing, claiming, authentication, organisations, search and public-page generation should be reused rather than rebuilt.

## Deployment requirement

After integration, Phase 2A must be deployed to the existing Render staging environment. Record:

- Frontend Render URL.
- Backend/API Render URL.
- Git branch.
- Exact deployed commit hash.
- Confirmation that the deployed commit contains the supplied Phase 2A implementation.
- Acceptance checklist results against the deployed version.
- Any small deployment-only changes or known issues.

# Phase 2B - Admin Automation & Management

## Objective

Add the management, audit, automation and operational controls needed once Phase 2A is stable.

## Core scope

- Import history and import error management.
- Duplicate reconciliation.
- In-app admin notifications.
- Audit log.
- Provenance / source tracking.
- Verification controls.
- Claim history.
- Listing completeness indicators.
- Website-source review queue.
- Advanced admin dashboard.
- Bulk actions.
- Advanced admin filters.
- Operational reporting.

## Availability management

Phase 2B expands the Normalized Availability Service with:

- Source configuration.
- Source priority.
- Multiple-source support.
- Conflict detection.
- Freshness thresholds.
- Fresh / aging / stale / unknown states.
- Manual overrides.
- Override expiry.
- Availability history.
- Source timestamps.
- Provider mappings.
- Sync status and monitoring.

A stale availability record must **not** automatically mean sold out.

## Phase 2 outcome

At the end of Phase 2, Deeparture should be able to build and maintain a large catalogue internally, invite operators to claim existing listings, manage enquiries and inventory, and prepare the platform for larger-scale supplier automation.

# Phase 3 - Operator Portal V2

## Objective

Give operators a modern self-service environment for maintaining the records Deeparture has already created for them.

## Core scope

- Operator dashboard.
- Organisation profile management.
- Manage claimed liveaboards.
- Manage claimed resorts.
- Manage itineraries and departures.
- Manage room types and packages.
- Update prices.
- Update availability.
- Create offers / discounts.
- Upload and manage images.
- Maintain facilities and attributes.
- Respond to enquiries.
- Claim additional existing listings where permitted.
- User / organisation membership and role management.
- Operator-facing notifications.
- Mobile-responsive workflow.

## Architectural principle

Operators should update existing Deeparture records whenever possible. Claiming must not create duplicate vessels, resorts or organisations.

## Future readiness

The same backend/domain services used by the Operator Portal should also support future supplier APIs and native applications.

# Phase 4 - Customer Frontend V2 + PWA

## Objective

Turn the V2 data platform into a polished public customer experience.

## Core scope

- Full customer-facing redesign.
- Modern liveaboard search.
- Modern resort search.
- Destination browsing.
- Date-based search.
- Effective-price filtering.
- Availability filtering.
- Vegan / dietary filters.
- Accessibility filters.
- Diving and facility filters.

- Detailed vessel and resort pages.
- Itinerary and departure pages.
- Resort package and seasonal pricing presentation.
- Enquiry / request-to-book flows.
- Mobile-first responsive design.
- SEO-friendly dynamic pages.
- PWA capability.

### Search principles

Filtering should occur at database/API level wherever practical rather than loading large result sets and filtering only in the browser.

### Pricing principles

- Retain native supplier currency.
- Allow normalized/search currency.
- Support itinerary base prices.
- Support departure overrides.
- Support offers/discounts.
- Display effective price consistently.

### Outcome

Deeparture becomes a modern searchable customer marketplace/catalogue rather than primarily an operator-entry backend.

## Phase 5 - Deeparture Supply Network & Data Automation

### Objective

Build Deeparture's own supplier-data network so the platform can receive, normalize and distribute catalogue, pricing and availability data from many sources.

This replaces the narrower idea of simply adding external API integrations. The aim is for Deeparture to become capable of operating its own liveaboard/resort data platform.

### Core scope

- Deeparture Supplier API.
- Webhook ingestion.
- Connector SDK / adapter framework.
- Booking-system connectors.
- Scheduled feed synchronization.
- Integration monitoring.
- JSON feeds.
- XML feeds.
- SFTP feeds.
- Google Sheets ingestion.
- Website ingestion / structured extraction.
- External provider connectors.
- Provider ID mapping.
- Catalogue synchronization.
- Price synchronization.
- Availability synchronization.
- Sync error handling.

- Retry / failure tolerance.
- Feed freshness monitoring.

### Architecture

All sources should feed the same normalized internal services:

```
Supplier / API / feed / connector / sheet / website
-> source adapter
-> normalized catalogue + availability services
-> Deeparture database
-> Admin / Operator / Customer / Agent / Partner API
```

### External platforms

Systems such as sharc may be connected as providers, but Deeparture should not depend on a single external provider.

### Commercial outcome

Deeparture can eventually act not only as a consumer of supplier data, but potentially as a structured liveaboard/resort data provider itself.

## Phase 6 - Agent Platform

### Objective

Extend Deeparture from a direct customer platform into a B2B distribution and agent environment.

### Core scope

- Travel-agent accounts.
- Agency organisations.
- Agency users and permissions.
- Agent search interface.
- Agent pricing.
- Net rates / commission structures.
- Agent enquiries and bookings.
- Saved trips / shortlists.
- Client quotations.
- Agent-specific availability.
- Booking/request management.
- Agency reporting.
- Supplier / agent communication workflow.
- Partner API readiness.
- White-label / embedded distribution options where commercially appropriate.

### Architecture

The Agent Platform should use the same catalogue, pricing, availability, organisation and enquiry services already used elsewhere in V2.

Avoid creating a separate parallel inventory or booking model specifically for agents.

### Outcome

Deeparture evolves into a multi-sided travel platform supporting:

- Deeparture Admin.
- Operators.

- Direct customers.
- Travel agents.
- Future partners / API consumers.

## Project-Wide Architectural Rules

These rules apply across all six phases.

- Use stable permanent internal IDs.
- Keep external provider IDs in mapping records rather than replacing internal IDs.
- Version APIs and import schemas.
- Use normalized catalogue and availability services.
- Make imports and synchronization idempotent.
- Prefer archive/soft-delete over destructive deletion.
- Keep pricing basis explicit.
- Retain native currency and optional normalized/search currency.
- Store source timestamps and received timestamps.
- Treat vessel/resort identity separately from current operator ownership where possible.
- Use structured workflow states rather than contradictory boolean flags.
- Preserve provenance and audit information.
- Keep business logic in backend/domain services rather than individual frontends.
- Maintain separate development/staging/production environments.
- Keep credentials and secrets outside catalogue records and source code.
- Design for background jobs, rate limits, provider failures and retries.
- Keep media, search and external integrations behind replaceable service boundaries.
- Preserve localization and privacy readiness.
- Avoid rebuilding the core when a future feature can be added through an adapter.

## Development Sequence

1. Phase 1 - Foundation & Staging - substantially complete.
2. Phase 2A - Core Admin V2 - current implementation/deployment stage.
3. Phase 2B - Admin Automation & Management.
4. Phase 3 - Operator Portal V2.
5. Phase 4 - Customer Frontend V2 + PWA.
6. Phase 5 - Deeparture Supply Network & Data Automation.
7. Phase 6 - Agent Platform.

Although Phase 2 is split operationally into 2A and 2B, it remains one of the six primary project phases.

Phase 2A should be deployed and accepted in Render staging before Phase 2B begins.

For every deployed phase, record:

- Git branch.
- Exact deployed commit hash.
- Frontend URL.
- Backend/API URL.
- Acceptance-test status.
- Any known issues or deployment-specific changes.